

Manipulations élémentaires avec Pandas

La librairie **Pandas** est la librairie dédiée à la manipulation de données en Python. Elle introduit deux structures clés :

- **Series** : Un tableau unidimensionnel étiqueté, similaire à une colonne dans une feuille de calcul.
- **DataFrame** : Une structure bidimensionnelle, tabulaire, avec des étiquettes de ligne et de colonne. C'est l'équivalent d'une base de données ou d'une feuille Excel en mémoire.

Assurez-vous que les librairies **pandas** et **numpy** sont installées :

1. Créez un DataFrame à partir d'un dictionnaire Python, représentant 3 étudiants (Nom, Filière, Note_Stats) comme suit :

```
import pandas as pd
import numpy as np

# 1. Importation: pd est l'alias standard
data = {
    'Nom': ['Alice', 'Bob', 'Charlie', 'Diana'],
    'Filiere': ['IA', 'Big Data', 'IA', 'Big Data'],
    'Note_Stats': [16.5, 12.0, 18.0, 14.5]
}
df_etudiants = pd.DataFrame(data)

# Affichage des 5 premieres lignes
print(df_etudiants.head())

# Information sur le DataFrame
print(df_etudiants.info())
```

2. Créez un fichier **grades.csv** contenant les données précédente et enregistrez-le.
3. Lisez ce fichier en utilisant la fonction **pd.read_csv()**.

```
# Enregistrement pour la simulation
df_etudiants.to_csv('grades.csv', index=False)

# Lecture du fichier
df = pd.read_csv('grades.csv')
print("\nDataFrame charge a partir du CSV:")
print(df.head())
```

4. Affichez la Série des notes de statistiques.
5. Affichez les lignes 1 à 3 (lignes d'index 1 et 2).

6. Affichez la valeur de la note du 3ème étudiant (ligne d'index 2, colonne d'index 2).

```
# 1. Selection d'une colonne (retourne une Serie)
notes = df['Note_Stats']
print("\nSerie des notes:\n", notes)

# 2. Selection par position (slice)
lignes_specifiques = df[1:3]
print("\nLignes 1 et 2:\n", lignes_specifiques)

# 3. Selection par position (.iloc[ligne, colonne])
note_charlie = df.iloc[2, 2]
print(f"\nNote de Charlie (index 2, colonne 2): {note_charlie}")
```

7. Créez un DataFrame contenant uniquement les étudiants de la filière 'IA'.
8. Créez un DataFrame contenant les étudiants 'IA' qui ont eu une note supérieure à 15.

```
# 1. Filtre simple
df_ia = df[df['Filiere'] == 'IA']
print("\nEtudiants de la filiere IA:\n", df_ia)

# 2. Filtre compose (necessite & pour AND, et les parentheses)
df_ia_top = df[(df['Filiere'] == 'IA') & (df['Note_Stats'] > 15)]
print("\nEtudiants IA ayant plus de 15:\n", df_ia_top)
```

9. Utilisez la méthode `.describe()` sur l'ensemble du DataFrame. Qu'observez-vous pour la colonne 'Filière'?
10. Calculez la moyenne et l'écart-type des notes de statistiques.
11. Calculez la moyenne des notes, séparément pour chaque filière ('IA' et 'Big Data').

```
# 1. Resume global
print("\nResume statistique complet:\n", df.describe())

# 2. Statistiques par colonne
moyenne_notes = df['Note_Stats'].mean()
std_notes = df['Note_Stats'].std()
print(f"\nMoyenne generale: {moyenne_notes:.2f}, Ecart-type: {std_notes:.2f}")

# 3. Statistiques par groupe (methode GROUP BY)
moyenne_par_filiere = df.groupby('Filiere')['Note_Stats'].mean()
print("\nMoyenne des notes par filiere:\n", moyenne_par_filiere)
```

Application : Analyse de logs

On se propose d'analyser les logs de performance d'un système de recommandation en production. L'objectif étant de s'assurer de la stabilité et de la fiabilité des métriques clés (Latence, Taux d'Erreur, etc.) avant son déploiement.

- **Outils :** `pandas`, `numpy`, `scipy.stats`, `seaborn`.

Génération du Jeu de Données

Créez un `DataFrame` `df_logs` de $N = 1000$ enregistrements comportant :

1. `Latence_ms` : Temps de réponse en millisecondes. Distribution Normale centrée $\mu = 50$, $\sigma = 10$.
2. `Taux_Erreur` : Pourcentage d'erreurs (0 à 1). Distribution Uniforme entre 0.01 et 0.10.
3. `Type_Modèle` : Catégorie du modèle utilisé, au choix parmi ['Classification', 'Régression', 'Génératif'] avec des probabilités respectives de [0.6, 0.3, 0.1].
4. `Mémoire_Go` : Mémoire allouée en Gigaoctets. Entiers aléatoires entre 2 et 16.

Filtration pour Détection d'Anomalies

Le seuil d'alerte pour la latence est de 65 ms.

1. Créez un sous-`DataFrame` `df_alertes` contenant uniquement les logs où la `Latence_ms` est supérieure à 65.
2. Calculez le pourcentage d'alertes par rapport au nombre total de logs.

Performance par Architecture

Calculez les métriques suivantes par `Type_Modèle` à l'aide de la méthode `.groupby()` :

1. La `Latence_ms moyenne` par type de modèle.
2. L'`écart-type` du `Taux_Erreur` par type de modèle.

Distribution de la Latence

Tracez un `sns.histplot` de la variable `Latence_ms` en superposant la courbe de densité (KDE).

La distribution est-elle visuellement symétrique? Cela suggère-t-il une loi Normale?

Matrice de Corrélation

Calculez la matrice de corrélation et affichez-la sous forme de `sns.heatmap`.

Test de Normalité (Shapiro-Wilk)

1. Appliquez le test de Shapiro-Wilk à la variable `Latence_ms` en utilisant `stats.shapiro()`.
2. Hypothèses: H_0 : La latence suit une loi Normale; H_1 : La latence ne suit pas une loi Normale.
3. Au seuil $\alpha = 5\%$, rejetez-vous H_0 ? Expliquez votre raisonnement basé sur la p-value.

Intervalle de Confiance pour le Taux d'Erreur

Estimez le vrai taux d'erreur du système en production avec un niveau de confiance de 99%.

Simulation du Déploiement

Ajoutez une colonne `Version_Modèle` ('V1' / 'V2'). Simulez une meilleure performance pour V2 (diminution du `Taux_Erreur`).

Comparaison Visuelle (Boxplot)

Utilisez un `sns.boxplot` pour comparer la distribution du `Taux_Erreur` en fonction de la `Version_Modèle`.

Validation Statistique (Test de Student)

Utilisez `stats.ttest_ind()` pour comparer les moyennes.